

**Build a complete test framework
covering login, cart, checkout
using advanced Playwright
techniques**

Objective

Build and deliver a complete, modular Playwright-based test automation framework that covers essential UI journeys, applies advanced Playwright features, and integrates with modern DevOps workflows.

Scenario

You've just taken on the role of lead test automation engineer for a fast-scaling e-commerce platform. Your task is not simply to write tests, but to build a production-grade test automation framework that validates business-critical flows such as login, add-to-cart, and checkout. Your solution must demonstrate the ability to detect functional issues and visual regressions early, run efficiently in CI/CD, and support long-term scalability and collaboration.

Instructions for learners

Start by setting up a well-structured Playwright project. Scaffold your tests into clear folders (e.g., tests/login, tests/cart) and configure Playwright with sensible defaults such as retries, parallel workers, headless mode, base URL, and timeout settings. All configuration should live in playwright.config.ts for reusability.

Write test scripts for the following user flows using semantic selectors, auto-waiting, and role-specific assertions where needed:



* Login with valid credentials and assert redirection to the user dashboard or homepage

* Add product(s) to cart and verify cart summary or item count update

* Proceed to checkout and assert successful order confirmation

Ensure these tests are built using advanced Playwright features such as browser contexts, storageState reuse, custom locators, and scoped interactions within specific UI containers.

Include at least one visual regression check using toHaveScreenshot() on a stable, consistent part of the UI (e.g., dashboard, cart summary, or checkout success page). Use it to detect layout changes, spacing shifts, or missing elements.

Configure parallel test execution to optimize runtime. Add environment-safe retries and auto-waiting behavior to reduce test flakiness across CI environments.

Integrate your framework with GitHub Actions as your CI tool. Set up a working `.github/workflows/playwright.yml` file that installs dependencies, installs browsers, runs tests, and uploads artifacts such as HTML reports, screenshots, and traces. Ensure the pipeline runs on pull requests and pushes to the main branch. Optionally, simulate Docker usage or mention how Docker containers could be used to isolate the test environment.

Project Deliverables

Submit a complete test automation repository or ZIP file containing:

→ Playwright test scripts for login, cart, and checkout flows with modular structure and semantic selectors

→ At least one visual regression assertion using `expect(page).toHaveScreenshot()`

→ A complete Playwright configuration file with retries, headless mode, baseURL, and parallelism

A working GitHub Actions CI pipeline file configured to run tests and upload artifacts

A README.md file containing setup instructions, prerequisites, CI instructions, and how to rerun tests or debug failures

Optional Dockerfile or docker-compose.yml if you choose to containerize the test suite or run it against a containerized web app

Peer Review Evaluation Criteria

After submission, you will be assigned two peer projects to review using a shared rubric. In each review, consider the following:

→ Are the tests complete and realistic for an actual e-commerce app

→ Is the test code modular, readable, and using Playwright best practices

→ Does the CI pipeline run smoothly, capture useful artifacts, and return clear pass/fail output

→ Are visual assertions placed meaningfully to catch layout regressions

Would the framework be maintainable and scalable by other engineers or teams

Tips for Success



Use your earlier hands-on labs as building blocks. They were designed to prepare you for this capstone. Prioritize test clarity, code readability, and maintainability. Add comments in your tests to explain selectors or workflows. Use Playwright's HTML report and trace viewer to debug and demonstrate value. Think like a lead engineer: design a solution that could be adopted by your team, integrated into daily workflows, and trusted in production.

Submission

Process



Complete your project locally or in a GitHub repository. Package it as a clean, structured ZIP or submit the repository link. Ensure that all instructions in the README are clear for any reviewer to set up and run the tests. Once submitted, complete the peer review for two other learners and reflect on your own submission based on their feedback.

Completion

By completing this capstone, you will have built an end-to-end Playwright test automation framework that reflects how automation is implemented at scale in modern product teams. This validates your ability to plan, write, run, and integrate meaningful automation that drives quality forward.